

浮點數行為實驗報告

Advanced C++ Homework 2

學號：113062331

姓名：林欣磊

繳交日期：2026/10/07

報告目的

透過可重現的程式、執行結果與組合語言，分析浮點型態的效能與精度、數學函式的實作、編譯器最佳化及十進位輸出行為。主要平台為 Intel 筆電的 WSL2 環境，另以 Apple M4 平台進行跨環境比較。

填寫方式

本文件為報告架構。請將待填欄位換成實測內容，插入截圖並完成分析；提交前刪除填寫提示。每題以實驗目的、方法、結果與原因分析呈現。題目細節與加分條件請以正式作業說明為準。

內容

Q0 實驗環境

Q1 不同浮點型態的效能與精度

Q2 數學函式實作

Q3 IEEE 754 與 fast math

Q4 浮點數輸出

Q5 跨執行環境比較

Q6 總結與額外浮點問題

Q7 AI 使用揭露

參考資料

Q0 實驗環境

0 1 主要與輔助平台

項目	主要平台（助手本機採集）	輔助平台（本人提供）
裝置	ASUS TUF Gaming F16 FX608JMR	MacBook Air（2025）
處理器	Intel Core i7-14650HX	Apple M4
架構／target	x86_64-linux-gnu	arm64-apple-darwin27.0.0
主機系統	Windows 11 家用版 25H2 Build 26200.9457	macOS Golden Gate 27.0.1 名稱與 build 待 sw_vers 核對
執行環境	WSL2／Ubuntu 26.04.1 LTS	macOS 原生環境
編譯器與版本	GCC 15.2.0 Ubuntu 15.2.0-16ubuntu1	Apple Clang 21.0.0 clang-2100.3.24.2
Q0 編譯參數	-std=c++20 -O2 -fno-fast-math -ffp-contract=off	尚未執行 Q0 型態程式
核心／執行緒	Windows：16／24 WSL 虛擬拓樸：12／24	待採集
記憶體	實體 32 GiB；WSL 約 15 GiB WSL swap 4 GiB	24 GB（本人提供）
WSL／kernel	WSL 2.7.3.0 6.6.114.1-microsoft-standard-WSL2	不適用／Darwin kernel 待採集
函式庫	glibc 2.43-2ubuntu2.4 libstdc++6 16-20260322-1ubuntu1	實際連結版本待採集
電源與採集日期	平衡；AC 電源；電池 100% 2026/10/04（Asia/Taipei）	資訊提供：2026/10/04 電源狀態待採集

WSL 的核心與快取資訊為虛擬環境呈現，不能直接當成實體 CPU 拓樸。Mac 的 g++ --version 回報 Apple Clang，與主要平台 GCC 不同；目前不符合 Q5 的同編譯器版本條件。

```
Windows PowerShell :  
./experiments/q0_environment/collect_windows.ps1  
WSL Ubuntu（專案根目錄）：  
bash experiments/q0_environment/collect_linux.sh results/q0-wsl  
Mac（專案根目錄）：  
bash experiments/q0_environment/collect_mac.sh
```

【截圖位置】0 1 CPU 作業系統與編譯器資訊
插入圖片後補上圖說，說明圖片支持的觀察。

0 2 浮點型態資訊

下表為 WSL GCC 的實際輸出。sizeof 表示儲存空間；long double 的 16 bytes 不等於 128-bit 有效精度。Mac 型態特性尚待實測。

型態	sizeof	digits	digits10	max_digits10	is_iec559
float	4	24	6	9	true
double	8	53	15	17	true
long double	16	64	18	21	true

WSL 補充輸出：radix=2；alignof 為 4／8／16 bytes；epsilon 為 2^-23／2^-52／2^-63；FLT_EVAL_METHOD=0；捨入模式為 FE_TONEAREST；fast-math 關閉。編譯器支援 _float128 擴充，但尚未測試其運算。

Q0 重現指令（WSL，專案根目錄）：
g++ -std=c++20 -O2 -fno-fast-math -ffp-contract=off experiments/q0_environment/main.cpp -o results/q0-wsl/q0_environment
./results/q0-wsl/q0_environment
原始紀錄：results/q0-windows/hardware.json、results/q0-windows/wsl-power.txt、results/q0-wsl/environment.txt。環境採集與型態程式由助手執行；本人仍須重跑、觀察並加入截圖。CPU 即時頻率、溫度與 Mac 完整採集尚待補充。

【截圖位置】0 2 型態資訊程式輸出
插入圖片後補上圖說，說明圖片支持的觀察。

Q1 不同浮點型態的效能與精度

1 1 實驗目的

使用相同演算法比較 float、double 與 long double 的執行時間及數值誤差。若另測 `_float128`，需記錄支援工具與額外編譯參數。

1 2 效能測試方法

準備測量 float、double、long double 的 scalar 加減、乘法及除法。每個 kernel 分為一條相依鏈與四條獨立鏈，每迭代分別為 2 與 8 個算術運算。初始值由執行期 seed 決定，`factor=1+1/1024`，配對運算維持值域。預設每組 2 次暖機（每次最多 100000 迭代）、7 次測量（每次 5000000 迭代）；使用 `steady_clock`，保留原始秒數、每運算時間、中位數與最小／最大值，輪換型態順序。

待測 kernel 獨立編譯，關閉 LTO、fast-math、FMA contraction 與自動向量化；`noinline` 與迴圈前後的 `optimization barrier` 防止跨函式簡化，每回合輸出 `checksum`。正式測量前檢查反組譯中仍有算術迴圈與分支。時間包含呼叫、少量參數準備、迴圈控制及收尾 `checksum` 成本；每運算時間不直接等於單條指令硬體 `latency`。

在 WSL Ubuntu 專案根目錄執行：
`bash experiments/q1_benchmark/build.sh`
`./build/q1-scalar/q1_benchmark --check`
`./build/q1-scalar/q1_benchmark --iterations 5000000 --repeats 7 --warmup 2 --seed 20261004`
`objdump -d -C build/q1-scalar/kernels.o`
編譯旗標：`-std=c++20 -O3 -Wall -Wextra -fno-fast-math -ffp-contract=off -fno-lto -fno-tree-vectorize`

1 3 效能結果

目前僅完成程式建置、18 條 kernel 路徑的不計時 smoke check 與組合語言檢查。尚未執行正式 benchmark 或精度實驗，以下結果表與截圖待本人執行後填入。

型態	Run 1	Run 2	Run 3	中位數
float	待填	待填	待填	待填
double	待填	待填	待填	待填
long double	待填	待填	待填	待填

若測量超過三次，保留完整原始紀錄，並於本表呈現摘要。

【截圖位置】1 1 Benchmark 原始輸出
插入圖片後補上圖說，說明圖片支持的觀察。

1 4 結果分析

【待填】解釋測試是在量測相依鏈的延遲、獨立運算的吞吐量，或整個演算法的時間。記錄是否向量化，避免把不同執行方式全歸因於型態精度。

Q1 精度與硬體分析

1 5 精度測試設計

精度測試使用 10000 組 $[2^{24}, 1, -2^{24}]$ 與 $[2^{53}, 1, -2^{53}]$ ，分別正序與反序加總。所有輸入皆可精確表示；實數精確答案為整數 10000，且三種型態都能精確表示。程式報告結果、絕對誤差與相對誤差，不假定 long double 為真值。執行：`./build/q1-scalar/q1_benchmark --precision-only`。

絕對誤差 = |實測值 - 參考值|
相對誤差 = 絕對誤差 / |參考值|，參考值為零時不使用此定義。

型態	計算結果	絕對誤差	相對誤差
float	待填	待填	待填
double	待填	待填	待填
long double	待填	待填	待填

【截圖位置】1 2 精度實驗輸出
插入圖片後補上圖說，說明圖片支持的觀察。

1 6 型態表示與指令

【待填】以實測 numeric_limits 資訊判斷有效精度，再用平台 ABI 或工具文件佐證表示方式。分別說明 float、double、long double。

【截圖位置】1 3 各型態運算核心的組合語言
插入圖片後補上圖說，說明圖片支持的觀察。

【待填】列出實際觀察到的指令或函式呼叫，說明它們如何支持效能差異的解釋。

1 7 小結

【待填】用一段文字總結本機測得的效能與精度取舍，並指出測試範圍。

Q2 數學函式實作

2 1 函式與規格

本節預定研究 `std::sqrt`。若改用其他 `cmath` 函式，請同步修改本節程式與分析。

【待填】記錄 `overload`、回傳型態、負數輸入與特殊值的行為，並引用規格來源。

2 2 實驗方法

```
#include <cmath>
float f(float x) { return std::sqrt(x); }
double g(double x) { return std::sqrt(x); }
long double h(long double x) { return std::sqrt(x); }

g++ -std=c++20 -O2 -S sqrt_test.cpp -o sqrt_test.s
```

2 3 本機實作路徑

【待填】追蹤 `std::sqrt` 經由標頭、`compiler builtin` 或 `libm` 到硬體指令的實際路徑。注意 `assembly` 可能同時包含硬體指令與處理錯誤的 `library fallback`。

【截圖位置】2 1 `sqrt` 的組合語言與分支
插入圖片後補上圖說，說明圖片支持的觀察。

2 4 型態比較

型態	指令或函式呼叫	觀察與來源
float	待填	待填
double	待填	待填
long double	待填	待填

2 5 小結

【待填】說明本平台如何執行 `sqrt`，以及哪些結論僅適用於目前版本與編譯參數。

Q3 IEEE 754 與 fast math

3 1 實驗目的

以相同程式比較一般最佳化與 `-ffast-math`，找出輸出差異及造成差異的具體轉換。

【待填】依 GCC 官方文件說明本實驗涉及的選項，例如 `reassociation`、有限值假設或 `signed zero`。FMA contraction 須另外確認相關參數與目標指令。

3 2 程式與編譯方式

【待填】放入完整關鍵運算，說明輸入、預期差異及如何避免只觀察到編譯期常數計算。

```
g++ -std=c++20 -O2 fast_math.cpp -o normal
g++ -std=c++20 -O2 -ffast-math fast_math.cpp -o fast
g++ -std=c++20 -O2 -S fast_math.cpp -o normal.s
g++ -std=c++20 -O2 -ffast-math -S fast_math.cpp -o fast.s
```

3 3 實驗結果

模式	輸出	關鍵指令或轉換
一般模式	待填	待填
fast math	待填	待填

【截圖位置】3 1 兩種模式輸出

插入圖片後補上圖說，說明圖片支持的觀察。

【截圖位置】3 2 與 3 3 兩種模式關鍵 assembly

插入圖片後補上圖說，說明圖片支持的觀察。

3 4 原因分析與小結

【待填】從原始運算順序、編譯器轉換、各步捨入到最終結果說明差異。根據實驗列出適用情境與需要保留的數值保證。

Q4 浮點數輸出

4 1 問題

printf("%.9f", x) 與 fixed 搭配 setprecision(9)，是否能精確描述為將原本的十進位數四捨五入到小數點後第九位？

4 2 反例與驗證

【待填】選擇反例，記錄十進位字面值、實際儲存值與格式化結果。區分字面值轉換的誤差與輸出格式化的捨入。

```
#include <cstdio>
#include <iomanip>
#include <iostream>
// 將下列 x 換成實驗選定的數值
// double x = ...;
// printf("%.9f\n", x);
// std::cout << std::fixed << std::setprecision(9) << x << "\n";
// std::cout << std::hexfloat << x << "\n";
```

觀察項目	結果
原始十進位字面值	待填
實際儲存值或十六進位表示	待填
printf 輸出	待填
iostream 輸出	待填
捨入模式與環境	待填

【截圖位置】4 1 反例輸出與儲存值
插入圖片後補上圖說，說明圖片支持的觀察。

4 3 結論

【待填】精確回答原敘述，說明二進位表示、十進位中點及 ties 的處理。若討論 rounding mode，提供實測或文件依據。

Q5 跨執行環境比較

5 1 實驗條件

本節為加分實驗。先確認正式題目對相同原始碼、編譯器版本、**flags** 與無未定義行為的要求。不同平台的 Q1 比較不一定直接滿足本題條件。

控制條件	Intel 平台	M4 平台
原始碼版本或雜湊	待填	待填
編譯器名稱與完整版本	待填	待填
編譯參數	待填	待填
OS 與函式庫	待填	待填
浮點環境與 ABI	待填	待填

5 2 程式与結果

【待填】放入實驗程式並解釋標準允許差異的依據，檢查是否依賴未定義行為。

【截圖位置】5 1 Intel 平台輸出
插入圖片後補上圖說，說明圖片支持的觀察。

【截圖位置】5 2 Apple M4 平台輸出
插入圖片後補上圖說，說明圖片支持的觀察。

5 3 原因分析

【待填】以具體證據判斷差異來自表示格式、指令、ABI、函式庫或浮點環境。若同時改變多個條件，說明哪些原因尚未被隔離。

5 4 小結

【待填】指出是否滿足加分条件，以及證據支持的結論。

Q6 總結與額外浮點問題

6 1 實驗總結

【待填】整合型態精度與效能、數學函式、最佳化及輸出的實驗發現。每項結論連結到本報告的結果或圖。

6 2 開發中的使用原則

【待填】說明如何依誤差容忍度選型態、處理運算順序與累積誤差、驗證最佳化。浮點比較應依用途選擇精確比較或適當誤差準則，不宜一律禁止 ==。

6 3 額外問題與實驗

候選主題為 catastrophic cancellation。若前面已使用此現象，請改選其他尚未討論的問題。比較以下兩個數學上等價的式子：

直接計算： $\text{sqrt}(x + 1) - \text{sqrt}(x)$

改寫計算： $1 / (\text{sqrt}(x + 1) + \text{sqrt}(x))$

【待填】選擇 x ，說明 $x + 1$ 是否可表示，以及相近數相減如何影響有效位數。

x	直接計算	改寫計算	參考值或誤差
待填	待填	待填	待填

【截圖位置】6 1 兩種公式的數值結果

插入圖片後補上圖說，說明圖片支持的觀察。

6 4 結論

【待填】根據實驗說明數學等價與數值穩定性之間的關係。

Q7 AI 使用揭露與參考資料

7 1 AI 使用揭露

請依实际使用情况修改下列文字，特别是尚未完成的執行與驗證部分。

本次作業使用 ChatGPT 與 Codex 輔助規劃實驗、撰寫測試程式、查找資料及整理報告。我自行執行的實驗包括【待填】，並透過【待填】驗證輸出與分析。AI 產生的程式或說明經由【待填】檢查與修正。

工具	協助內容	本人驗證方式
ChatGPT	待填	待填
Codex	待填	待填

參考資料

以下為待查證的來源類別。完成查閱後，填入文件名稱、版本或章節、網址與查閱日期，並在相關段落標示引用編號。

- [1] C++ 標準或標準函式庫文件 【待填】
- [2] GCC 最佳化與 fast math 官方文件 【待填】
- [3] IEEE 754 或相關數值表示文件 【待填】
- [4] Intel 指令集文件 【待填】
- [5] 本機 libm 或標準函式庫原始碼 【待填】
- [6] ARM 或 Apple 平台文件 【依實驗需要填寫】

繳交前確認

所有結果均有原始紀錄；圖片可讀且附圖說；編譯與執行指令完整；參考資料可追溯；AI 揭露符合實際使用情況；刪除待填欄位與填寫提示。